

Requirements Elicitation & Use-Case Engineering

Student Study Edition · Golden Master design · 10 structured lessons with original course visuals, examples, activities, and study aids.

01 · Requirements Elicitation

02 · Why Requirements Are Hard — and What We Must Specify

03 · The Requirements Elicitation Process

04 · Information Collection & Analysis

05 · Use-Case Foundations

06 · Derive, Specify & Trace Use Cases

TOPIC 05 · STUDY MAP

Contents

Use this map to move through the topic in the same lesson order as The Educator course pages.

01	Requirements Elicitation	3
02	Why Requirements Are Hard — and What We Must Specify	7
03	The Requirements Elicitation Process	10
04	Information Collection & Analysis	13
05	Use-Case Foundations	23
06	Derive, Specify & Trace Use Cases	28
07	Use-Case Relationships	32
08	Designing Clear Use-Case Diagrams	34
09	Requirements Specification & Feasibility	36
10	Requirements Validation, Reviews & Agile Practice	38

TOPIC 05 · LESSON 01 · 1 OF 10 LESSONS

TOPIC 05 · REQUIREMENTS & USE-CASE ENGINEERING · LESSON 01

Requirements Elicitation

The hardest question is often not how to build the software, but **what exactly should be built—and why?**

LEARNING FOCUS

Explain why elicitation matters and distinguish a business problem, a stakeholder wish, and a software requirement.

PRACTICE LAB · TOPIC 05**Mission 05 — Model the Real-Estate Experience**

Elicit and classify requirements for BaytLink, identify actors and actor goals, inspect flawed UML use-case drafts, diagnose boundary and relationship defects, and choose stronger redesigns across 16 stages.

One continuous engineering story

Chapter 7 is now integrated here instead of repeated as a separate topic. Follow the chain: **stakeholder reality → requirements → use-case foundations → derive & validate actor goals → specify them → prove traceability → refine the UML model.**

[Open the integrated derivation lesson →](#)

THE CENTRAL IDEA

From stakeholder reality to an agreed specification

- ✓ **Requirements** are capabilities the system must deliver.
- ✓ **Requirements elicitation** identifies and formulates the real capabilities and constraints for the future software system.
- ✓ Requirements are documented in a specification that may form part of a contract.
- ✓ The planning phase includes elicitation, deriving use cases, and producing an iterative development plan.
- ✓ The team works closely with customers and users to understand needs, determine real requirements, and prioritize them.

Importance of Requirements Elicitation



Source: Jim Johnson, "Chaos: Charting the Seas of Information Technology." The Standish Group, 1994.



Visual anchor. Elicitation starts in the messy real world—problems, goals, users and constraints—and turns that understanding into software commitments.

WHY IT MATTERS

Requirements mistakes multiply downstream

Contract

A requirements specification can carry legal and commercial consequences.

Rework

A late discovery can force redesign, recoding and retesting.

Project risk

The lecture uses the Standish CHAOS study to emphasize how strongly requirements problems can affect projects.

Challenges of Requirements Elicitation

— Misunderstandings between Analyst and Stakeholder —



Clear **communication** and **collaboration** are key to successful requirements elicitation.



Read the historical percentage as lecture context, not a timeless constant. The durable lesson is the high downstream cost of requirements defects.

WORKED EXAMPLE

Clinic scheduling

Problem: patients struggle to find appointments. **Wish:** “make booking easier.”

Requirement: “The system shall let a patient search available appointments by specialty, physician and date and confirm a selected slot.”

REMEMBER

Elicitation does not record wishes verbatim. It discovers the underlying need and converts it into requirements precise enough to design, build and test.

TOPIC 05 · LESSON 02 · 2 OF 10 LESSONS

TOPIC 05 · CHAPTER 4 · LESSON 02

Why Requirements Are Hard — and What We Must Specify

Stakeholders speak in goals, frustrations and wishes; engineers must turn them into precise capabilities, qualities and constraints.

LEARNING FOCUS

Explain the major elicitation challenges and distinguish functional from non-functional requirements.

WHY ELICITATION FAILS

Different people hold different pieces of the problem

- ✓ Customers, users and developers have different backgrounds, terminology and assumptions.
- ✓ The team may not know the application domain; customers may not know what software can realistically do.
- ✓ Needs are incomplete, conflicting and change throughout the lifecycle.
- ✓ Software development is often a **wicked problem**: learning more can change how the problem itself is understood.
- ✓ Non-functional requirements are often understated even though they may determine usability, security, safety or scalability.

Communication Barrier – Misunderstanding



Poor communication can lead to **confusion**, **incorrect assumptions**, and **failed outcomes**.



Common Effects

- Confusion
- Wrong Assumptions
- Rework & Delays
- Project Failure



How to Overcome



Ask Questions



Active Listening



Confirm Understanding



Use Common Language



Document & Validate



Bridging the communication gap builds shared understanding and successful solutions.

Visual anchor. Words can sound precise while stakeholders still mean different things. Good elicitation verifies shared meaning.

TWO REQUIREMENT FAMILIES

What the system does vs. how well and under what limits

⚙️ Functional requirements

Statements of the **information-processing capabilities** the software must possess.

Official example: a car-rental system shall let a customer inquire about available cars using make, model, dates, price range and class.

🛡️ Non-functional requirements

Performance, quality, safety, security and interface requirements, plus relevant design or implementation constraints.

Performance

Quality

Safety

Security

Interface

OFFICIAL EXAMPLES

Make qualities measurable where possible

Performance

Handle 10,000 simultaneous inquiries;
response time ≤ 3 seconds under typical
workload.

Quality & constraints

Protect user privacy and website contents;
support specified platforms, databases and
user-interface channels.

WORKED CLASSIFICATION

University admissions

Functional: applicants upload documents and submit an application. **Performance:** 95% of page requests complete within 2 seconds under 5,000 concurrent users. **Security:** only authorized admissions staff may view confidential documents.

MEMORY HOOK

FUNCTIONAL = CAPABILITY. NON-FUNCTIONAL = QUALITY OR CONSTRAINT.

Classify by engineering intent, not by keywords alone.

TOPIC 05 · LESSON 03 · 3 OF 10 LESSONS

TOPIC 05 · CHAPTER 4 · LESSON 03

The Requirements Elicitation Process

Elicitation is repeated learning: collect evidence, model it, derive requirements, test feasibility, and review the result.

LEARNING FOCUS

Explain the five major elicitation steps and the purpose of each one.

OFFICIAL LECTURE STRUCTURE

The five major steps

1 · Collect

Gather information about the application, business, users, environment, problems, goals and constraints.

2 · Model

Build formal or informal analysis models when they improve shared understanding.

3 · Derive

Turn evidence, goals, needs, standards and policies into requirements and constraints.

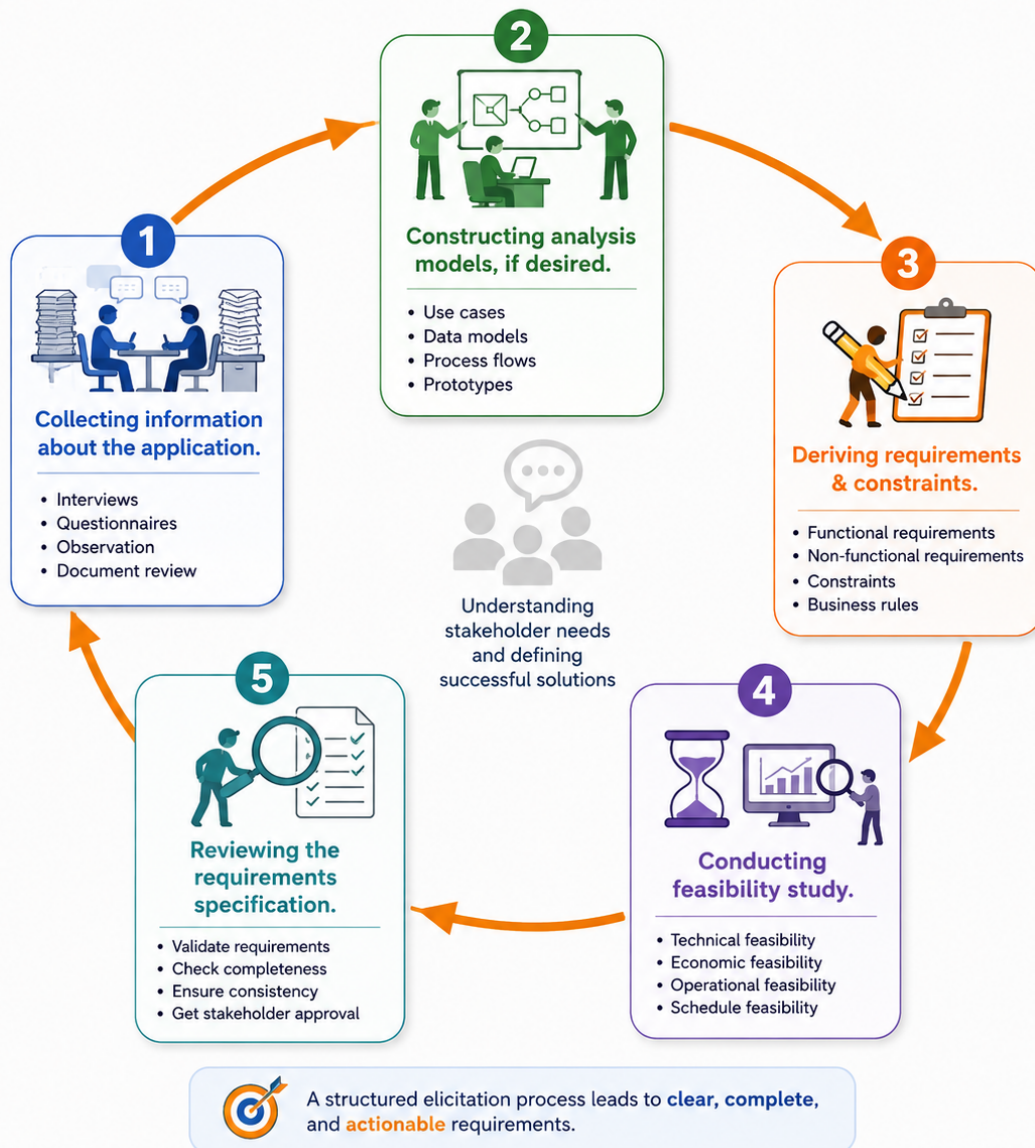
4 · Feasibility

Check whether requirements are practical under technology, time, cost and external constraints.

5 · Review

Check the specification for correctness, completeness, consistency and agreement.

Requirements Elicitation Steps



Process map. Study the steps in order, but remember that real elicitation loops backward whenever new evidence exposes misunderstanding or infeasibility.

WORKED EXAMPLE

Clinic scheduling

Collect: interview receptionists and observe booking. **Model:** map appointment flow. **Derive:** formulate search, booking and notification requirements. **Feasibility:** check hospital identity and SMS integration. **Review:** walk through the requirements with users and technical staff.

PRIORITIZATION

Requirements compete for value, time and cost

The lecture uses willingness-to-pay and desired delivery timing to expose priorities. The wider principle is to make trade-offs explicit rather than treating every request as equally mandatory.

REMEMBER

COLLECT → MODEL → DERIVE → FEASIBILITY → REVIEW. The next lesson opens up the first three steps in detail.

TOPIC 05 · LESSON 04 · 4 OF 10 LESSONS

TOPIC 05 · CHAPTER 4 · LESSON 04

Information Collection & Analysis

Strong requirements come from **multiple evidence sources**, domain understanding and models that expose hidden assumptions.

LEARNING FOCUS

Select suitable information sources, investigate the domain, and turn evidence into requirements and constraints.

COLLECTION TECHNIQUES

Use more than one window into reality

 Customer presentation

Goals, terminology and scope.

 Literature survey

Domain knowledge and known practices.

 Forms & procedures

Real data, workflow and controls.

 Regulations & standards

Mandatory external constraints.

 Stakeholder survey

Structured input at scale.

 User interview

Motivations, exceptions and tacit knowledge.

 Observation

What people actually do.

 User stories

Role, need and value in concise form.

Information Collection Techniques

Different ways to gather the right information for better decisions



Visual anchor. Elicitation is multi-source; one interview is never the whole system.

UNDERSTAND THE DOMAIN

Four question groups organize the investigation

Business & context

How does the business operate? What surrounds the system?

Processes & problems

What flows, inputs, outputs, delays and workarounds exist?

People & goals

Who are the users? What outcomes and priorities matter?

Quality & constraints

What performance, security, standards and policy limits apply?

Businesses in Different Domains

Diverse industries. Unique needs. Smarter solutions.



Domain knowledge changes the questions you ask: finance, healthcare, transportation, education and other domains have different rules, risks and language.

ANALYSIS MODELS

Give scattered evidence a shape stakeholders can challenge

Formal or informal models help the team understand the application, requirements and constraints. Their purpose is understanding—not decoration.

Constructing Analysis Models

Building models to understand applications, requirements, and constraints



Intuitive Models

Easy-to-understand, conceptual representations



Physical Models

Represent the physical structure or environment of the system.



Process Models

Show how processes transform inputs into outputs.



Stakeholder Models

Identify people, roles, and their interactions with the system.



Conceptual Models

High-level view of key concepts and their relationships.



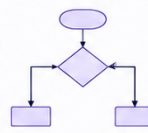
Purpose: to aid **understanding** of the application, requirements, and constraints.



Formal or Informal Models

Structured representations for precise analysis

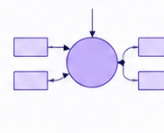
Flowcharts



Show control flow and decision points.

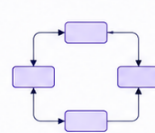
Formal Models

Data Flow Diagrams



Represent data movement between processes.

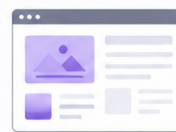
State Diagrams



Model states and transitions.

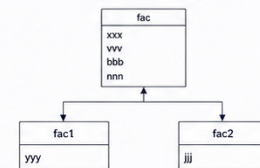
Informal Models

Prototypes / Mockups



Visual representation of the system for early feedback.

Class / Structure Diagrams



Show structure, classes, and relationships.



Well-constructed models lead to **better understanding**, clearer requirements, and **successful solutions**.



Model the evidence so assumptions, missing information and disagreement become visible.

DERIVE REQUIREMENTS

Requirements come from evidence, goals and constraints

Current reality

Problems, goals, needs, procedures and policies.



Analysis

Clarify, reconcile, model and identify constraints.



SRS commitments

Functional requirements, qualities, interfaces and constraints.

Deriving Requirements and Constraints



Requirements are **derived, not invented**: current conditions, business goals, needs, problems, standards and policies converge into the SRS.

EXTENDED WORKED EXAMPLE · FROM STORY TO SPECIFICATION

SkyGate Adventure Park — deriving requirements from a real operational story

Imagine a large fantasy-themed adventure park. Guests buy tickets online, arrive with families, enter through smart gates, reserve popular quests and rides, receive live waiting-time updates, purchase food and merchandise, and use a mobile app to navigate the park. Some attractions have age, height or health restrictions. Parents may link children to a family account. Operators temporarily close attractions because of weather or technical problems, while emergency staff must quickly locate affected zones and communicate instructions. Today, several activities are disconnected: ticketing uses one system, ride reservations another, restaurants use separate point-of-sale terminals, and staff sometimes announce

closures manually. Guests complain about duplicate queues, missed reservations and discovering restrictions only after reaching an attraction. Management therefore proposes the **SkyGate Experience System (SES)**: one integrated platform connecting guests, gates, attractions, staff and operational services.



1 · Plan

A family buys tickets and creates linked guest profiles before the visit.



2 · Enter

Smart gates validate admission while preserving each guest's entitlements.



3 · Adventure

Guests discover quests, check restrictions and reserve attraction slots.



4 · Adapt

The app reacts to queues, closures and schedule changes with useful alternatives.



5 · Operate safely

Staff monitor capacity, incidents and attraction status from one operational view.

How do we derive requirements from that story?

STEP	EVIDENCE / QUESTION	ANALYSIS ACTION	REQUIREMENT OUTCOME
 1. Discover	Interview guests, operators, safety staff and management; observe gates and queues.	Separate stated wishes from recurring operational problems.	Candidate needs and stakeholder goals.
 2. Understand	Why are reservations missed? Who controls restrictions? What happens during closure?	Map actors, workflows, rules, exceptions and external systems.	Domain model and system boundary.
 3. Derive functions	What services must SES provide to achieve each goal?	Convert goals and tasks into observable system behavior.	Functional requirements (FR).

STEP	EVIDENCE / QUESTION	ANALYSIS ACTION	REQUIREMENT OUTCOME
 4. Derive qualities	How fast, secure, available, usable and scalable must those services be?	Attach measurable quality expectations and constraints.	Non-functional requirements (NFR).
 5. Check	Is each statement necessary, feasible, unambiguous and testable?	Review with stakeholders and define measurable acceptance evidence.	Validated candidate requirements for the SRS.

FUNCTIONAL · what the system shall do

10 Functional Requirements

- 1 FR-01 — Account & profile:** The SES shall allow a guest to create an account and maintain individual guest profiles, including linked child profiles managed by an authorized parent or guardian.
- 2 FR-02 — Ticket purchase:** The SES shall allow guests to select admission products, dates and quantities, complete payment, and receive a unique digital admission credential for each eligible guest.
- 3 FR-03 — Gate admission:** The SES shall validate a guest's digital credential at an entrance gate and record successful entry while rejecting

NON-FUNCTIONAL · how well / under what constraint

10 Non-Functional Requirements

- 1 NFR-01 — Performance:** The SES shall return attraction availability and estimated waiting-time information within 2 seconds for at least 95% of requests under the defined normal operating load.
- 2 NFR-02 — Gate performance:** The admission-validation service shall return an accept/reject result to a connected smart gate within 1 second for at least 99% of validation requests under normal operating conditions.
- 3 NFR-03 — Availability:** Guest admission, attraction status and reservation services shall provide at least 99.9%

invalid, expired or already-consumed admission credentials according to park policy.

4 FR-04 — Attraction discovery: The SES shall present available attractions with location, operating status, estimated waiting time and applicable age, height or health restrictions.

5 FR-05 — Eligibility checking: Before confirming a reservation, the SES shall compare the selected guest profile with the attraction's recorded eligibility rules and inform the guest when a restriction prevents participation.

6 FR-06 — Reservation: The SES shall allow an eligible guest to reserve an available attraction time slot and shall prevent the same guest from holding conflicting reservations.

7 FR-07 — Change notification: When an

availability during published park operating hours, excluding approved maintenance windows.

4 NFR-04 — Scalability: The SES shall support at least 25,000 concurrently active guest sessions without violating the specified response-time requirements.

5 NFR-05 — Security: The SES shall require authenticated, role-based authorization for staff functions so that operational, safety and administrative actions are accessible only to approved roles.

6 NFR-06 — Privacy: The SES shall protect stored and transmitted personal data using organization-approved encryption and shall restrict access to child-profile information to authorized accounts and staff roles.

7 NFR-07 — Reliability: A confirmed reservation shall not be lost or duplicated following a

attraction is delayed, suspended or closed, the SES shall identify affected reservations and notify the corresponding guests of the change.

8 FR-08 — Alternative recommendation: When a reserved attraction becomes unavailable, the SES shall present currently available alternative attractions or time slots that are compatible with the guest's recorded eligibility.

9 FR-09 — Staff operations: The SES shall allow authorized operators to update attraction operating status, capacity restrictions, temporary closures and reopening information.

10 FR-10 — Incident support: The SES shall allow authorized safety staff to record an operational incident, identify the affected attraction or zone, and distribute an approved alert to relevant staff and affected guests.

recoverable service interruption; reservation transactions shall preserve consistency after recovery.

8 NFR-08 — Usability: A first-time guest shall be able to locate an attraction and view its current wait time and eligibility restrictions within three primary navigation actions from the app home screen.

9 NFR-09 — Accessibility: Guest-facing web and mobile interfaces shall conform to the accessibility standard adopted by the organization for production digital services.

10 NFR-10 — Auditability: The SES shall retain a timestamped audit record of staff changes to attraction status, capacity, restrictions and incident alerts, including the authenticated staff identifier associated with each change.

STUDY THE DISTINCTION

FR ASKS “WHAT OBSERVABLE SERVICE OR BEHAVIOR MUST THE SYSTEM PROVIDE?” NFR ASKS “HOW WELL MUST IT WORK, OR WHAT QUALITY/CONSTRAINT MUST IT SATISFY?” Notice that both types are written as complete, testable statements—not one-word labels such as “security” or “performance.”

REMEMBER

EVIDENCE → UNDERSTANDING → MODEL → REQUIREMENT. The next lesson shows one important way to express high-level functional goals: use cases.

TOPIC 05 · LESSON 05 · 5 OF 10 LESSONS

TOPIC 05 · CHAPTER 4 · LESSON 05

Use-Case Foundations

A use-case model asks: **which external role wants what useful result from the system?**

LEARNING FOCUS

Read and construct UML use-case diagrams using actors, use cases, system boundaries and relationships.

WHAT THE MODEL CAPTURES

Goals, interactions and required behavior

- ✓ A **use-case diagram** depicts a useful subset of the larger use-case model for communication.
- ✓ A **use-case model** describes user goals, interactions with the system, and behavior required to satisfy those goals.
- ✓ Core elements are **actors, use cases, the system boundary and relationships**.
- ✓ Use cases support requirements communication, analysis/design discovery, and test identification.

VISUAL ANCHOR · READ FROM OUTSIDE IN

Actor → goal → system responsibility



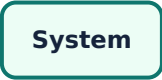
Adventure

RealmQuest Adventure System**Accept Quest****Equip Artifact****Enter Dungeon****Claim Reward**

The actor is outside because it is an external role; the use cases are inside because they are services supplied by the system.

NOTATION REFERENCE

Use-case diagram elements and symbols

Element	Notation	Meaning	Rule
Actor		External role: person, organization, device or system.	Name the role; keep it outside the boundary.
Use Case		A useful system behavior that achieves an actor goal.	Use a clear verb-object goal name.
System Boundary		The system/subsystem being modeled.	Put applicable use cases inside and actors outside.
Association		Actor participates in a use case.	Use a solid line.
«include»		Required reused behavior.	Arrow points to the included use case.
«extend»		Optional/conditional behavior added to a base use case.	Arrow points from extension to base.
Generalization		Specialization among actors or use cases.	Triangle points to the general parent.

OFFICIAL LECTURE EXAMPLE FIRST

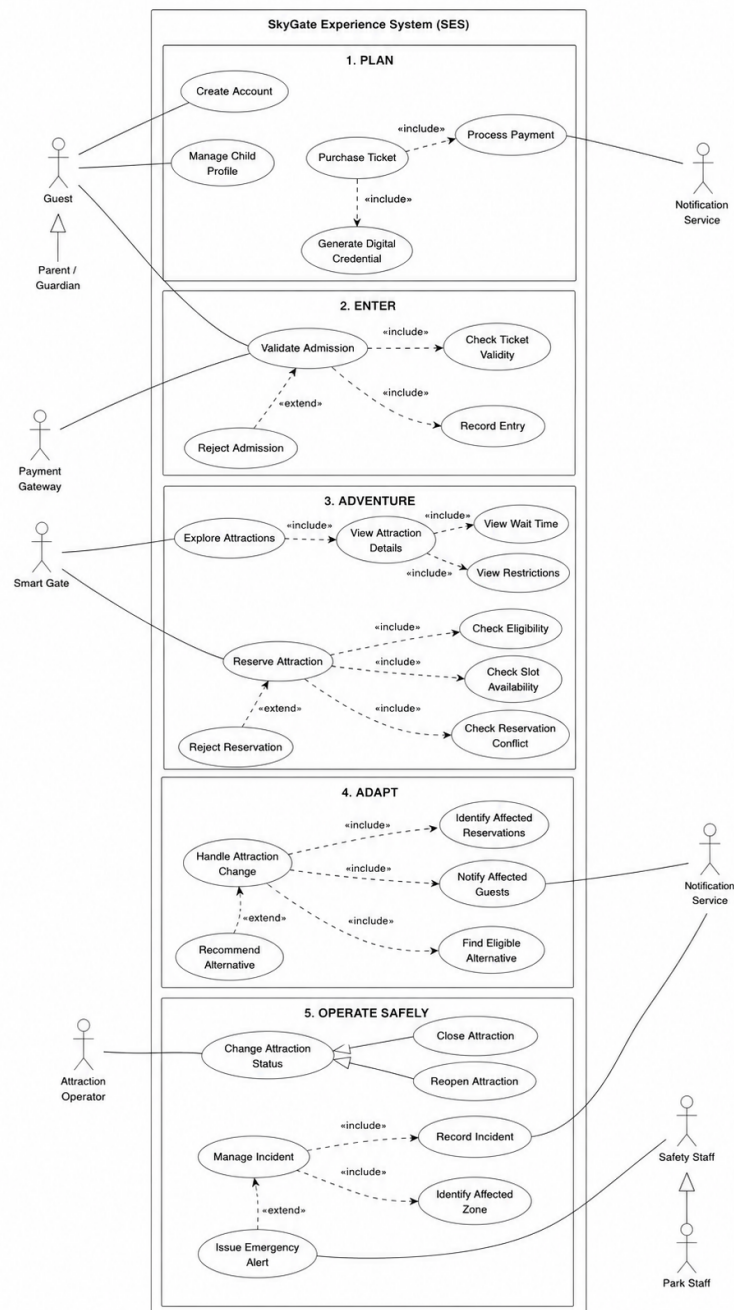
Library system

Actor: Patron. **System:** Library System. **Use cases:** Check Out Document, Return Document, Search for Document. This is the official baseline example from the lecture.

APPLIED MODEL · CONTINUING THE SKYGATE CASE

From 10 functional requirements to a use-case diagram

The previous lesson derived the SkyGate Experience System requirements from the park story. This diagram turns those functional responsibilities into an actor-centered view so you can see **who interacts with the system, which goals belong inside the system boundary, and how related behavior is connected.**



SkyGate Adventure Park — full UML use-case view. Select the diagram to open it at its original resolution.

1 · Start with the actors

Read the diagram from outside the system boundary. Each actor represents an external role, not a specific person.

2 · Find each actor's goals

Follow the associations into the boundary. Each connected oval should describe a useful result the system provides.

3 · Read relationship semantics

If «include» or «extend» appears, read the label and arrow direction carefully: include is required reuse;

4 · Trace back to requirements

Use cases mainly organize **functional requirements**. Performance, security, availability and

extend is optional or conditional behavior.

other NFRs usually constrain these use cases rather than becoming separate use cases.

EXTERNAL UML EXAMPLES · UML-DIAGRAMS.ORG

See the notation in real systems

External source visual: Bank ATM use case diagram · <https://www.uml-diagrams.org/examples/use-case-example-atm.png>

Featured · Bank ATM

Use it to identify primary/secondary actors, system boundary and transaction goals.

Open source ↗

Explore 4 more examples

External source visual: e-Library use case diagram · <https://www.uml-diagrams.org/examples/use-case-example-library-opac.png>

e-Library / OPAC **Open source** ↗

External source visual: Online shopping use case diagram · <https://www.uml-diagrams.org/examples/use-case-example-online-shopping.png>

Online Shopping **Open source** ↗

External source visual: Point of sale use case diagram · <https://www.uml-diagrams.org/examples/use-case-example-pos.png>

Point of Sale **Open source** ↗

External source visual: Hospital reception use case diagram · <https://www.uml-diagrams.org/examples/use-case-example-hospital-reception.png>

Hospital Reception **Open source** ↗

MEMORY HOOK

**ACTOR = OUTSIDE ROLE. USE CASE = USEFUL GOAL. BOUNDARY = SYSTEM
SCOPE. RELATIONSHIP = WHY ELEMENTS CONNECT.**

TOPIC 05 · LESSON 06 · 6 OF 10 LESSONS

TOPIC 05 · CHAPTER 7 INTEGRATION

Derive, Specify & Trace Use Cases

Do not invent use cases from a blank diagram. **Derive them from requirements, validate the actor goal, choose the right specification depth, then prove coverage.**

LEARNING FOCUS

Turn requirement statements into defensible use cases and connect every use case back to an agreed requirement.

THE COMPLETE TRANSFORMATION

Requirement → candidate → valid use case → specification → traceability



Requirement

Start from agreed capabilities.



Extract

Look for domain verb-noun tasks.



Validate

Test the actor-centered business goal.



Specify

Choose the needed detail level.



Trace

Prove requirements coverage.

1 · FINDING CANDIDATES

Mine requirements for domain goals—not implementation actions

Look for domain-specific **verb-noun phrases** that mean “do something,” “something must be done,” or “perform some task.” Then identify the actor and the system/subsystem that owns the behavior.

Requirement phrase

Candidate

Decision

Guest shall **reserve an attraction**

Reserve Attraction

✓ Goal

Operator shall **update attraction status**

Update Attraction Status

✓ Goal

System shall **update database row**

Update Database Row

✗ Internal step

User shall **press Submit**

Press Submit

✗ UI action

2 · VALIDATION GATE

A candidate becomes a use case only if all four answers are YES

① Business process?

Is it a complete domain task rather than a click or algorithm?

② Begins with actor?

Can an external role initiate the business process?

③ Ends with actor?

Does the interaction return to the actor with an observable outcome?

④ Useful to actor?

Does it accomplish something valuable for that role?

ALL FOUR MUST BE YES.

“Query Database,” “Search Linked List,” and “Press Submit Button” may be implementation steps, but they are not complete actor goals.

3 · USE-CASE SPECIFICATION

One goal, three levels of abstraction

① Abstract

Reserve Attraction

A concise verb-noun phrase naming the business process.

② High Level

Start → goal achieved

TUCBW: the guest selects an available attraction slot.

TUCEW: the guest sees a confirmed reservation.

③ Expanded

Actor ↔ System steps

Describe the interaction step by step, including alternatives and exceptions when needed.

High-level rule: describe what begins the interaction and what useful result ends it. Do not fill the high-level use case with database updates or other background processing.

4 · REQUIREMENTS-USE CASE TRACEABILITY

Coverage must be visible, not assumed

A traceability matrix answers two directions: **Does every functional requirement have use-case coverage?** and **Does every use case have a requirement/business justification?**

SkyGate requirement	Purchase Ticket	Validate Admission	Reserve Attraction	Update Status	Notify Guest
FR-02 Ticket purchase	●	—	—	—	—
FR-03 Gate admission	—	●	—	—	—
FR-06 Reservation	—	—	●	—	—
FR-07 Change notification	—	—	—	—	●
FR-09 Staff operations	—	—	—	●	—

DIAGNOSTIC POWER

A requirement row with no mapping suggests a **MISSING USE CASE**. A use-case column with no requirement may indicate **SCOPE CREEP, AN UNDOCUMENTED NEED, OR AN UNNECESSARY USE CASE**. **Priority/weight** can also be carried into the matrix to guide development order.

SKYGATE SYNTHESIS

See the chain as one engineering argument

FR-06: “The SES shall allow an eligible guest to reserve an available attraction time slot...”

↓ extract domain task

Candidate: Reserve Attraction

↓ validate: business process ✓ actor starts ✓ actor receives outcome ✓ useful goal ✓

High-level use case: TUCBW the guest selects an available slot → TUCEW the guest sees a confirmed reservation.

↓ trace

FR-06 ↔ UC Reserve Attraction

MASTER RULE

**REQUIREMENTS JUSTIFY USE CASES; ACTORS GIVE THEM PURPOSE;
SPECIFICATIONS GIVE THEM DETAIL; TRACEABILITY PROVES COVERAGE.**

TOPIC 05 · LESSON 07 · 7 OF 10 LESSONS

TOPIC 05 · CHAPTER 4 · LESSON 06

Use-Case Relationships

A relationship is useful only when it explains **participation, mandatory reuse, optional extension, or specialization**.

LEARNING FOCUS

Choose association, «include», «extend», or generalization from scenario semantics and draw the direction correctly.

FOUR MEANINGS

Learn the logic before the arrow



Association

Actor —
Use Case

Actor
participates in
the use case.



«include»

Base - ->
Included

Required
reused
behavior
needed to
complete the
base use case.



«extend»

Extension -
-> Base

**Optional/
conditional**
behavior
added to a
base use case
that is
meaningful by
itself.



Generalization

Specific —>
General

A specialized
actor or use case
inherits from a
more general
parent.

OFFICIAL EXAMPLES

Required vs. optional is the crucial distinction

«include» · ATM

Withdraw Cash and **Transfer Cash**
can include **Verify Balance** because the
reused behavior is necessary.

«extend» · Purchase

Print Receipt can extend **Purchase**
Book with Credit Card when a receipt
is requested.

DECISION TEST

Which relationship fits?

Actor interacts with goal?
Association

Common behavior must execute?
«include»

Extra behavior is optional?
«extend»

One element specializes another?
Generalization

MEMORY HOOK

INCLUDE = MUST. EXTEND = MAY. GENERALIZATION = IS-A. ASSOCIATION = INTERACTS. «extend» does not mean “happens later.”

TOPIC 05 · LESSON 08 · 8 OF 10 LESSONS

TOPIC 05 · CHAPTER 4 · LESSON 07

Designing Clear Use-Case Diagrams

A good diagram is a **communication view**, not a warehouse for every actor and relationship.

LEARNING FOCUS

Design focused use-case diagrams that communicate scope and goals without unnecessary complexity.

OFFICIAL GUIDELINES

Clarity beats completeness on one canvas

 Avoid

- Too many use cases in one diagram.
- Many one-use-case diagrams.
- Crossing links and unnecessary relationships.
- Actors, databases or implementation objects inside the boundary.

 Prefer

- Coherent groups of closely related use cases.
- Only actors and goals relevant to the current view.
- Meaningful system/subsystem names.
- Generalization only when it reduces repetition.

OFFICIAL SAMS LESSON

Decompose one overloaded diagram into focused views

**Authentication**

Start Up,
Shutdown, Log
In, Log Out.

**Program
Management**

Add, Delete, Edit
Program.

**User
Management**

Create, Delete,
Update User.

**End-User
Services**

Search, Display
Detail, Apply
Online.

This applies two lecture principles: **separation of concerns** and **divide and conquer**.

BOUNDARY RULE

Inside = system behavior; outside = external roles

Inside

Use cases and their relationships.

Outside

Human actors, devices and external systems/services.

Actor link

Actor–use case links are associations.

Subsystem name

A meaningful name can later suggest a package/module boundary.

SUBMISSION CHECK

Is the boundary named? Are actors outside? Are goals clear? Are only relevant elements shown? Does every «include», «extend» or generalization have a defensible meaning?

TOPIC 05 · LESSON 09 · 9 OF 10 LESSONS

TOPIC 05 · CHAPTER 4 · LESSON 08

Requirements Specification & Feasibility

The SRS records **what is agreed**; feasibility asks whether the team can responsibly commit to it.

LEARNING FOCUS

Recognize the major SRS sections and evaluate feasibility across technology, schedule, cost and external constraints.

SRS STRUCTURE

Organize requirements so they can be reviewed and traced

1 · Introduction

Purpose, scope, definitions, references, overview.

2 · General Description

Context, functions, users, constraints, assumptions and dependencies.

3 · Specific Requirements

Functions, interfaces, performance, design constraints, quality attributes and other requirements.

4 · Appendices

Supporting material needed to interpret the specification.

INSIDE SPECIFIC REQUIREMENTS

Know the major categories

Functional

Inputs, processing and outputs for required capabilities.

Performance

Workload, response time and throughput obligations.

External interfaces

User, hardware, software and communication interfaces.

Constraints & qualities

Standards, hardware limits, security, availability, maintainability and related attributes.

FEASIBILITY

Desirable does not automatically mean doable



Technical

Can technology satisfy the functionality, performance and quality constraints?



Schedule

Can it be delivered in the required time?



Cost & resources

Is it affordable with available people and infrastructure?



External constraints

Do customer policy, industry standards or government rules permit it?

WORKED EXAMPLE

Face-recognition check-in

The capability may be technically possible yet infeasible if privacy regulation blocks the proposed data use, required accuracy cannot be achieved, cameras exceed budget, or integration cannot be completed before launch.

REMEMBER

SPECIFICATION RECORDS THE COMMITMENT; FEASIBILITY PROTECTS THE COMMITMENT FROM BEING UNREALISTIC. A requirement can be clear and testable yet still infeasible.

TOPIC 05 · LESSON 10 · 10 OF 10 LESSONS

TOPIC 05 · CHAPTER 4 · LESSON 09

Requirements Validation, Reviews & Agile Practice

Requirements become trustworthy when they survive **technical, domain and customer scrutiny**—and when detail is created at the right time.

LEARNING FOCUS

Distinguish verification from validation, compare review perspectives and techniques, and explain agile requirements practice.

VERIFICATION VS. VALIDATION

Two different quality questions

✂ Verification

Are we producing the requirements/models correctly? Check consistency, completeness, form and conformance.

🎯 Validation

Are these requirements for the right product? Check whether they reflect what customers and users actually need.

THREE REVIEW PERSPECTIVES

No single reviewer sees every defect

🛠 Technical review

Internal technical team examines engineering quality and defects.

🧠 Expert review

Domain experts check terminology, rules and assumptions.

👤 Customer/User review

Customers and users check real goals, tasks and priorities.

TECHNICAL TECHNIQUES

Peer review, walkthrough and inspection

👥 Peer Review

🚶 Walkthrough

🔍 Inspection

A checklist targets defects such as incompleteness,

Peers examine requirements, often guided by a questionnaire.

The analyst explains requirements while reviewers question assumptions.

duplication, inconsistency and ambiguity.

AGILE REQUIREMENTS PRACTICE

Enough detail for the current decision

 **Customer collaboration**

 **High-level early requirements**

 **Prioritize progress**

 **Just-barely-enough modeling**

Agile does **not** mean “no requirements.” It means avoiding premature detail that is likely to change and elaborating the next slice when it becomes relevant.

TOPIC 05 SYNTHESIS

The complete requirements story

- ✓ Discover the business problem and stakeholder needs.
- ✓ Separate capabilities from qualities and constraints.
- ✓ Collect evidence and model the domain.
- ✓ Express actor goals with use cases when appropriate.
- ✓ Derive and prioritize requirements, then check feasibility.
- ✓ Organize commitments in the SRS and review them from technical, domain and customer perspectives.
- ✓ Refine as understanding changes.

CONCEPT CHECK

Can you answer these without looking back?

- 1**
Functional vs. non-functional?
- 2**
«include» vs. «extend»?
- 3**
Verification vs. validation?
- 4**
Why can a clear requirement still be infeasible?
- 5**
Why use multiple information sources and review perspectives?

FINAL MEMORY HOOK

ELICIT → UNDERSTAND → MODEL → SPECIFY → CHECK → LEARN → REFINE.